

Chapter 7: Direct Methods

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

MIT Press

Chapter 7 Overview

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

What are Direct Methods?

Direct methods (also called *zero-order*, *black-box*, *pattern search*, or *derivative-free* methods) rely **solely on the objective function f** .

- Do *not* use gradient or Hessian information
- Suitable when derivatives are unavailable or unreliable (e.g. simulation outputs)
- Use other criteria to choose the next search direction and to judge convergence

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.1 Cyclic Coordinate Search — Concept

Definition

Cyclic coordinate search (coordinate descent / taxicab search) alternates line searches along the n coordinate directions, optimising one variable at a time while holding the rest fixed.

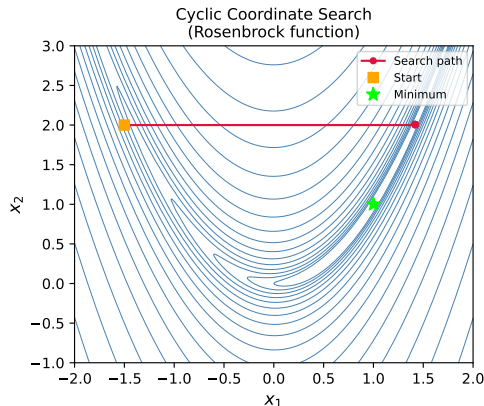
Update equations:

$$x_2^{(1)} = \arg \min_{x_2} f(x_1^{(1)}, x_2, x_3^{(1)}, \dots, x_n^{(1)}) \quad (7.1)$$

$$x_3^{(2)} = \arg \min_{x_3} f(x_1^{(1)}, x_2^{(1)}, x_3, \dots, x_n^{(1)}) \quad (7.2)$$

This is equivalent to a sequence of line searches along the n *basis vectors*:

$$\begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}, \dots, \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix} \quad k = i$$



Cyclic coordinate search on Rosenbrock: path alternates between coordinate directions.

7.1 Cyclic Coordinate Search — Algorithm

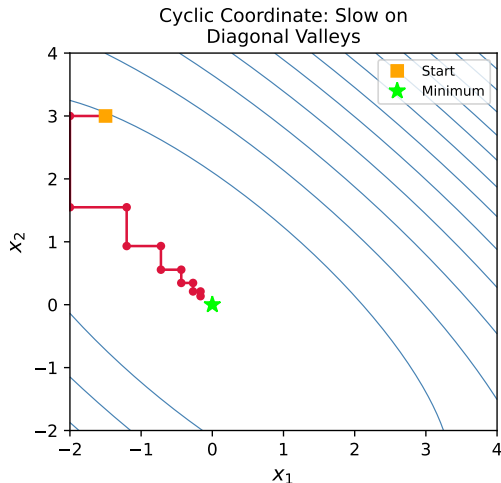
Algorithm 7.1 — Cyclic Coordinate Search

Input: f , $\mathbf{x}^{(1)}$, tolerance ϵ

- 1 Set $\Delta = \infty$, $n = \text{length}(\mathbf{x})$
- 2 **while** $\Delta > \epsilon$:
- 3 $\mathbf{x}' \leftarrow \mathbf{x}$
- 4 **for** $i = 1, \dots, n$:
- 5 $d \leftarrow \mathbf{e}^{(i)}$
- 6 $\mathbf{x} \leftarrow \text{line_search}(f, \mathbf{x}, d)$
- 7 $\Delta \leftarrow \|\mathbf{x} - \mathbf{x}'\|$
- 8 **return** $\mathbf{x}$

Acceleration

Add a step along $\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ after each full cycle to exploit momentum:



Problem: cyclic search can be very slow on diagonally-oriented

7.1 Cyclic Coordinate Search — Python

Listing 1: Cyclic coordinate search in Python (NumPy/SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize_scalar
3
4 def cyclic_coordinate_search(f, x0, eps=1e-6, max_cycles=200):
5     """
6     Cyclic coordinate search (coordinate descent with line search).
7     f : objective function  $\mathbb{R}^n \rightarrow \mathbb{R}$ 
8     x0 : initial point (array-like)
9     """
10    x = np.array(x0, dtype=float)
11    n = len(x)
12    for _ in range(max_cycles):
13        x_prev = x.copy()
14        for i in range(n):
15            # Line search along the i-th coordinate direction
16            def fi(t):
17                xc = x.copy()
18                xc[i] = t
19                return f(xc)
20            result = minimize_scalar(fi, bracket=(x[i]-2, x[i]+2),
21                                         method='brent')
22            x[i] = result.x
23        if np.linalg.norm(x - x_prev) < eps:
24            break
25    return x
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method**
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.2 Powell's Method — Motivation

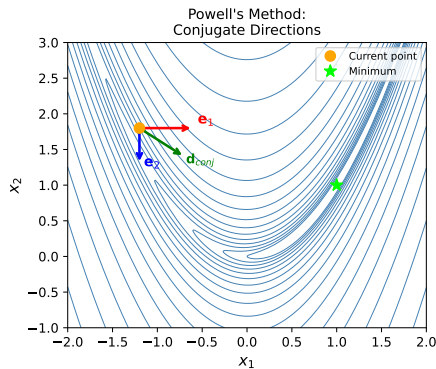
Key Idea

Powell's method searches along directions that are **non-orthogonal** and *mutually conjugate* to exploit the geometry of the objective. Unlike cyclic coordinate search it can move *diagonally*.

Conjugate directions: Two directions $\mathbf{u}$ and $\mathbf{v}$ are *conjugate* w.r.t. $\mathbf{Q}$ if

$$\mathbf{u}^\top \mathbf{Q} \mathbf{v} = 0$$

For a quadratic $f = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{b}^\top \mathbf{x}$, Powell's method finds the minimum in at most n iterations.



Powell's method builds conjugate search directions from coordinate directions.

7.2 Powell's Method — Algorithm

Algorithm 7.4 — Powell's Method

Input: f , start $\mathbf{x}$, tolerance ϵ

- 1 $U \leftarrow [\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(n)}]$ (basis directions)
- 2 $\Delta \leftarrow \infty$
- 3 **while** $\Delta > \epsilon$:
 - 4 $\mathbf{x}' \leftarrow \mathbf{x}$
 - 5 **for** $i = 1, \dots, n$:
 - 6 $d \leftarrow U[i]$
 - 7 $\mathbf{x} \leftarrow \text{line_search}(f, \mathbf{x}, d)$
 - 8 **for** $i = 1, \dots, n - 1$:
 - 9 $U[i] \leftarrow U[i + 1]$
 - 10 $U[n] \leftarrow \mathbf{x} - \mathbf{x}'$ (new conjugate direction)
 - 11 $\mathbf{x} \leftarrow \text{line_search}(f, \mathbf{x}', U[n])$
 - 12 $\Delta \leftarrow \|\mathbf{x} - \mathbf{x}'\|$
- 13 **return** $\mathbf{x}$

Key Property

A new search direction is formed as:

$$\mathbf{d}^{(\text{new})} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$$

This direction tends to be **conjugate** to all previous search directions, so progress accumulates without re-doing work.

Convergence Note

Powell's method automatically adjusts step sizes via line search. It decreases the step size once no improvements are found ($\Delta < \epsilon$).

7.2 Powell's Method — Python

Listing 2: Powell's method (Python/SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize_scalar, minimize
3
4 def powell(f, x0, eps=1e-8, max_iter=200):
5     """
6     Powell's method for unconstrained minimisation.
7     f : callable R^n -> R
8     x0 : initial point (1-D array)
9     """
10    x = np.array(x0, dtype=float)
11    n = len(x)
12    U = np.eye(n) # start with coordinate directions
13
14    for _ in range(max_iter):
15        x_prev = x.copy()
16        for i in range(n):
17            d = U[i]
18            res = minimize_scalar(lambda t: f(x + t * d))
19            x = x + res.x * d
20        # Shift directions and add conjugate direction
21        U[:-1] = U[1:]
22        U[-1] = x - x_prev
23        norm = np.linalg.norm(U[-1])
24        if norm > 1e-12:
25            d = U[-1]
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method**
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.3 Hooke-Jeeves Method — Concept

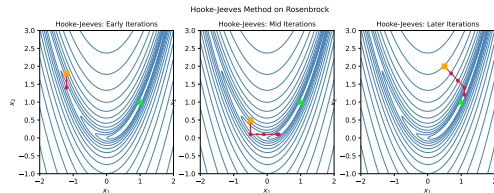
Definition

The **Hooke-Jeeves method** (1961) traverses the search space using small steps in each coordinate direction from an *anchoring point*. Two-phase iteration:

- 1 **Exploratory search:** evaluate $f(\mathbf{x})$ and $f(\mathbf{x} \pm \alpha \mathbf{e}^{(i)})$ for each dimension i ; keep improvements.
- 2 **Pattern move:** if the exploratory step improved, take an additional step in the same direction:

$$\mathbf{x}_{\text{pattern}}^{(k+1)} = \mathbf{x}^{(k+1)} + (\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)})$$

If no improvement is found, reduce the step size $\alpha \leftarrow \alpha/2$.



Hooke-Jeeves on Rosenbrock: starts with large steps, reduces step size when stuck.

7.3 Hooke-Jeeves Method — Algorithm

Algorithm 7.5 — Hooke-Jeeves

Input: f , $\mathbf{x}$, initial step α , tolerance ϵ

- 1 **while** $\alpha > \epsilon$:
- 2 $\mathbf{x}^{(\text{base})} \leftarrow \mathbf{x}$
- 3 **for** $i = 1, \dots, n$:
- 4 Try $\mathbf{x} + \alpha \mathbf{e}^{(i)}$ and $\mathbf{x} - \alpha \mathbf{e}^{(i)}$
- 5 Keep whichever reduces f most; else keep $\mathbf{x}$
- 6 **if** improvement found:
- 7 Pattern step: $\mathbf{x} \leftarrow \mathbf{x} - \mathbf{x}^{(\text{base})}$
- 8 **else**:
- 9 $\alpha \leftarrow \alpha/2$
- 10 **return** $\mathbf{x}$

Numerical Example (2D)

Let $f(x_1, x_2) = x_1^2 + 2x_2^2$, start at $\mathbf{x} = [2, 1]$, $\alpha = 0.5$:

- Try $[2.5, 1]$: $f = 8.25$ (worse)
- Try $[1.5, 1]$: $f = 4.25$ (better) ✓
- Try $[1.5, 1.5]$: $f = 6.75$ (worse)
- Try $[1.5, 0.5]$: $f = 2.75$ (better) ✓
- New point: $[1.5, 0.5]$
- Pattern move: $[1.0, 0.0]$

Convergence

Provably converges to a local minimum on *continuous* functions. Method is susceptible to local minima.

7.3 Hooke-Jeeves — Python

Listing 3: Hooke-Jeeves method in Python

```
1 import numpy as np
2
3 def hooke_jeeves(f, x0, alpha=1.0, eps=1e-6):
4     """
5     Hooke-Jeeves (pattern search) method.
6     f      : callable  $R^n \rightarrow R$ 
7     x0     : initial point
8     alpha  : initial step size
9     eps    : stopping tolerance (step size)
10    """
11    x = np.array(x0, dtype=float)
12    n = len(x)
13    while alpha > eps:
14        x_base = x.copy()
15        x_trial = x.copy()
16        improved = False
17        for i in range(n):
18            for sign in (1, -1):
19                x_new = x_trial.copy()
20                x_new[i] += sign * alpha
21                if f(x_new) < f(x_trial):
22                    x_trial = x_new
23                    improved = True
24                break
25        if improved:
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search**
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.4 Generalized Pattern Search — Overview

Key Idea

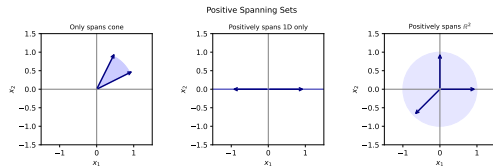
Generalized pattern search (GPS) generalises Hooke-Jeeves by allowing *any* set of directions $\mathcal{D} = \{\mathbf{d}^{(1)}, \dots, \mathbf{d}^{(m)}\}$ as long as they form a **positive spanning set**.

Positive Spanning Set

A set $\mathcal{D}$ is a *positive spanning set* for $\mathbb{R}^n$ if every vector in $\mathbb{R}^n$ can be written as a **non-negative** combination of vectors in $\mathcal{D}$:

$$\forall \mathbf{v} \in \mathbb{R}^n, \exists \lambda_i \geq 0 : \mathbf{v} = \sum_i \lambda_i \mathbf{d}^{(i)}$$

Requires $m \geq n + 1$ directions.



Left: spans only a cone. Centre: spans 1D only. Right: positively spans $\mathbb{R}^2$.

7.4 Generalized Pattern Search — Algorithm & Mesh

Algorithm 7.6 — Generalized Pattern Search

Input: f , $\mathbf{x}$, initial step α , directions $\mathcal{D}$, tolerance ϵ

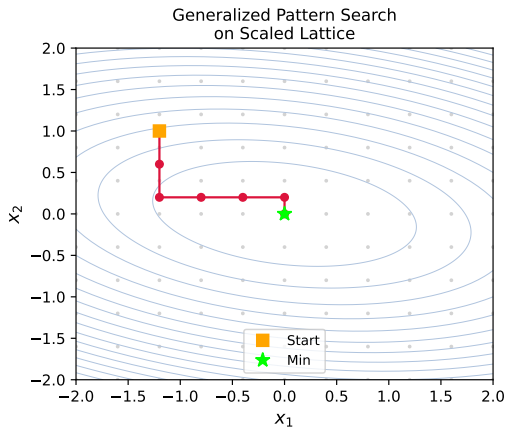
- 1 **while** $\alpha > \epsilon$:
- 2 **for** each $\mathbf{d} \in \mathcal{D}$:
- 3 **if** $f(\mathbf{x} + \alpha\mathbf{d}) < f(\mathbf{x})$:
- 4 $\mathbf{x} \leftarrow \mathbf{x} + \alpha\mathbf{d}$; **break**
- 5 **if** no improvement:
- 6 $\alpha \leftarrow \alpha/2$
- 7 **return** $\mathbf{x}$

Mesh Structure

All evaluated points lie on a scaled *lattice* (mesh):

$$\mathbf{x}^{(k)} = \mathbf{x}^{(1)} + \alpha^{(k)} \mathbf{D} \mathbf{z}, \quad \mathbf{z} \in \mathbb{Z}_{\geq 0}^m$$

The lattice need not be axis-aligned.



All previous points in GPS lie on a scaled lattice. The mesh is not explicitly stored.

7.4 Generalized Pattern Search — Python

Listing 4: Generalized pattern search in Python

```
1 import numpy as np
2
3 def generalized_pattern_search(f, x0, alpha=1.0, eps=1e-6,
4                               directions=None):
5     """
6     GPS with a user-supplied positive spanning set.
7     Defaults to +-e_i (2n coordinate directions).
8     """
9     x = np.array(x0, dtype=float)
10    n = len(x)
11    if directions is None:
12        # Default: coordinate directions + their negatives
13        directions = np.vstack([np.eye(n), -np.eye(n)])
14
15    while alpha > eps:
16        improved = False
17        for d in directions:
18            x_trial = x + alpha * d
19            if f(x_trial) < f(x):
20                x = x_trial
21                improved = True
22                break
23        if not improved:
24            alpha *= 0.5
25    return x
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method**
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.5 Nelder-Mead — Simplex Concept

What is a Simplex?

A **simplex** in $\mathbb{R}^n$ has $n + 1$ vertices:

- $n = 1$: line segment (2 points)
- $n = 2$: triangle (3 points)
- $n = 3$: tetrahedron (4 points)

Notation

Sort simplex vertices by objective value:

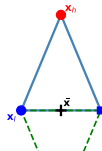
$$y_l \leq y_s \leq \dots \leq y_h$$

where:

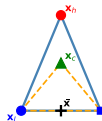
- $\mathbf{x}_l$ — **lowest** (best) vertex
- $\mathbf{x}_s$ — second highest
- $\mathbf{x}_h$ — **highest** (worst) vertex

Nelder-Mead Simplex Operations

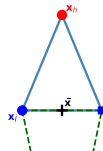
Reflection



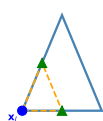
Contraction



Expansion



Shrinkage



Four Nelder-Mead operations: reflection, expansion,
contraction, shrinkage

7.5 Nelder-Mead — Four Operations

Reflection ($\alpha > 0$, typically $\alpha = 1$)

$$\mathbf{x}_r = \bar{\mathbf{x}} + \alpha(\bar{\mathbf{x}} - \mathbf{x}_h)$$

Reflects worst point over centroid. Moves simplex from high to lower regions.

Expansion ($\beta > \max(1, \alpha)$, typically $\beta = 2$)

$$\mathbf{x}_e = \bar{\mathbf{x}} + \beta(\mathbf{x}_r - \bar{\mathbf{x}})$$

Sends reflected point even further (when reflection was very good: $y_r < y_l$).

Contraction ($\gamma \in (0, 1)$, typically $\gamma = 0.5$)

$$\mathbf{x}_c = \bar{\mathbf{x}} + \gamma(\mathbf{x}_h - \bar{\mathbf{x}})$$

Shrinks simplex toward centroid, away from worst point

Shrinkage

All points except $\mathbf{x}_l$ move toward the best point:

$$\mathbf{x}^{(i)} \leftarrow \frac{\mathbf{x}^{(i)} + \mathbf{x}_l}{2}, \quad i \neq l$$

Used when contraction also fails.

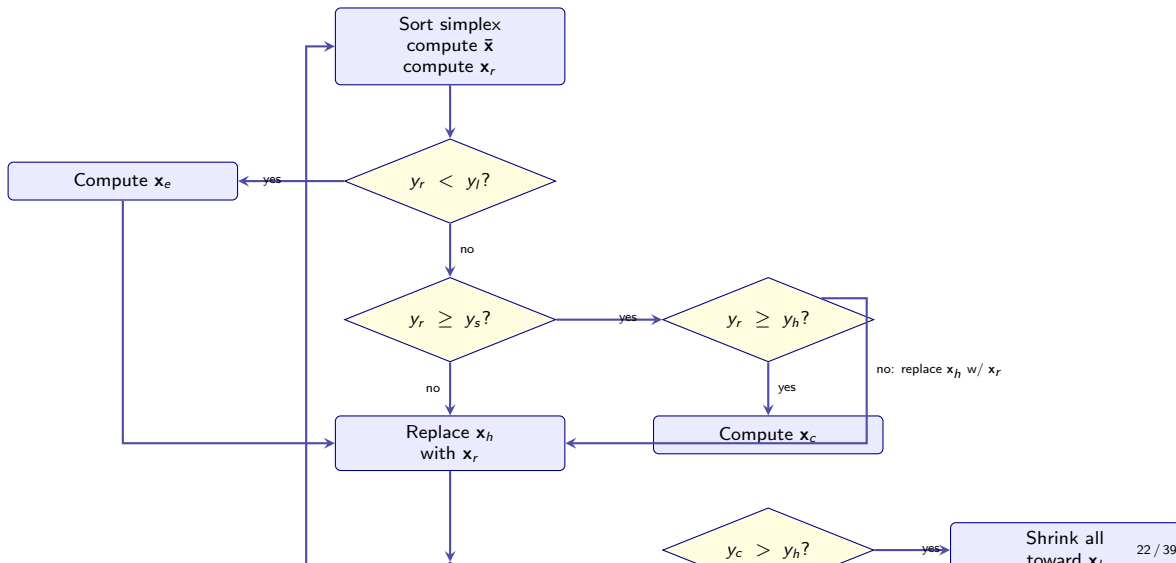
Convergence Criterion

Stop when standard deviation of function values is small:

$$\Delta = \text{std}(y^{(1)}, \dots, y^{(n+1)}) < \epsilon$$

High Δ = curved region (more work

7.5 Nelder-Mead — Decision Flowchart



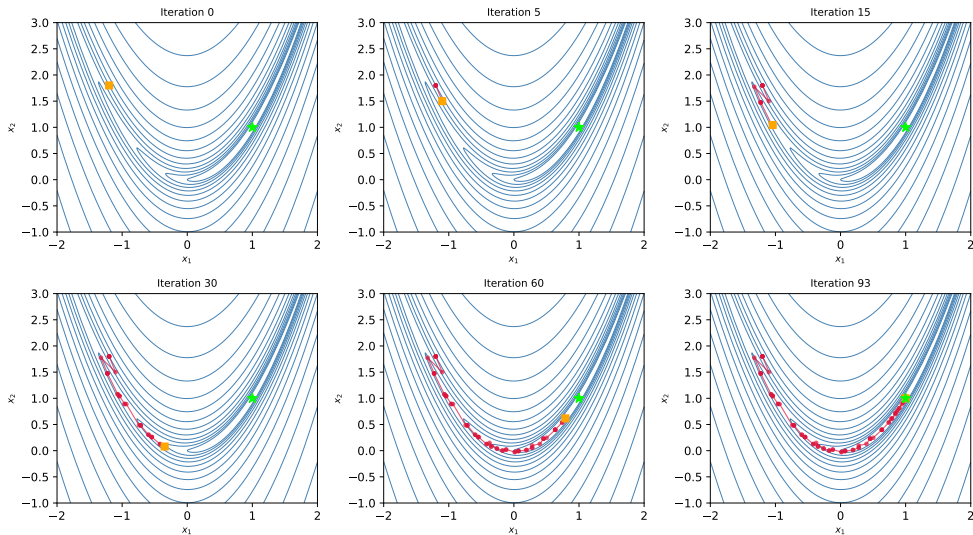
7.5 Nelder-Mead — Python

Listing 5: Nelder-Mead simplex method (pure Python + SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def nelder_mead(f, simplex, eps=1e-6, alpha=1.0, beta=2.0, gamma=0.5):
5     """
6     Nelder-Mead simplex method.
7     f : callable R^n -> R
8     simplex : list/array of (n+1) initial vertices, shape (n+1, n)
9     """
10    S = [np.array(s, dtype=float) for s in simplex]
11    while True:
12        y = np.array([f(s) for s in S])
13        p = np.argsort(y)
14        S = [S[i] for i in p]; y = y[p]
15        if np.std(y) < eps:
16            break
17        xl, yl = S[0], y[0]
18        xh, yh = S[-1], y[-1]
19        xs, ys = S[-2], y[-2]
20        xm = np.mean(S[:-1], axis=0) # centroid
21        xr = xm + alpha * (xm - xh) # reflection
22        yr = f(xr)
23        if yr < yl: # try expansion
24            xe = xm + beta * (xr - xm)
25            S[-1] = xe if f(xe) < yr else xr
```

7.5 Nelder-Mead — Iterations Visualised

Nelder-Mead on Rosenbrock (12 iterations shown)



Nelder-Mead on the Rosenbrock function. The simplex gradually migrates from the upper left toward the

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)**
- 7 Summary & Exercises

7.6 DIRECT — Overview

Motivation

DIRECT (Dlvided RECTangles) is a global optimisation algorithm for *Lipschitz-continuous* functions on a *bounded* domain $[\mathbf{a}, \mathbf{b}]$. It avoids the need to specify the Lipschitz constant ℓ .

Lipschitz Condition

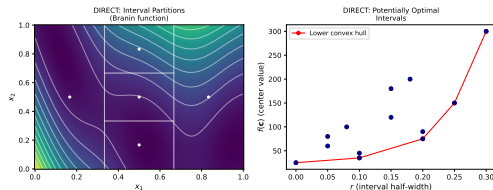
For all $\mathbf{x}, \mathbf{y}$ in the domain:

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq \ell \|\mathbf{x} - \mathbf{y}\|$$

A lower bound at point $\mathbf{c}$ with radius r is:

$$f(\mathbf{x}) \geq f(\mathbf{c}) - \ell r \quad \text{for all } \mathbf{x} \text{ in interval}$$

Optimisation Goal



Left: DIRECT interval partitions on the Branin function.
Right: $(r, f(\mathbf{c}))$ scatter with lower convex hull identifying potentially optimal intervals.

7.6 DIRECT — Potentially Optimal Intervals

Key Concept

An interval I with center $\mathbf{c}$ and radius r is **potentially optimal** if there exists some $\ell \geq 0$ such that:

- 1 $f(\mathbf{c}) - \ell r \leq f(\mathbf{c}') - \ell r'$ for all other intervals I'
- 2 $f(\mathbf{c}) - \ell r \leq f_{\min} - \epsilon |f_{\min}|$

This identifies intervals on the **lower-right convex hull** of the $(r, f(\mathbf{c}))$ scatter plot.

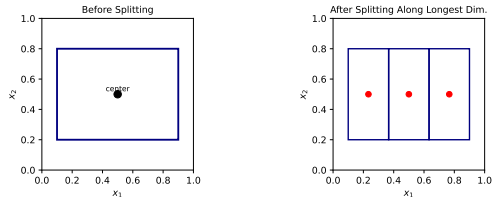
Splitting Rule

An interval is split along its *longest* dimension(s) into **three equal parts**. The two new center points are evaluated.

$$r_{\text{new}} = \frac{r}{3}$$

Splits along the axis with *smallest* f value are done first.

DIRECT: Rectangle Splitting



DIRECT splits the longest dimension into thirds. Width decreases by factor 1/3 each split.

Width After k Splits

In n dimensions, after k total splits on dimension i (k_i times each):

$$r = \sqrt[n]{\sum_{i=1}^n \left(\frac{1}{2} \cdot \frac{1}{2^{k_i}} \right)^2}$$

7.6 DIRECT — Algorithm

Algorithm 7.8 — DIRECT

Input: f , bounds $\mathbf{a}$, $\mathbf{b}$, max iterations $k_{\max}$, min radius $r_{\min}$

- 1 Normalise: $g(\mathbf{x}') = f(\mathbf{x}' \cdot (\mathbf{b} - \mathbf{a}) + \mathbf{a})$
- 2 Initialise single interval centered at $\mathbf{c} = [0.5, \dots, 0.5]$
- 3 Evaluate $g(\mathbf{c})$, $r = \sqrt{n} \cdot 0.5$
- 4 **for** $k = 1, \dots, k_{\max}$:
 - 5 Find *potentially optimal* intervals $\mathcal{S}_{\text{split}}$
 - 6 Remove them from $\mathcal{S}$
 - 7 **for** each interval in $\mathcal{S}_{\text{split}}$:
 - 8 Split and evaluate new centers
 - 9 Add sub-intervals to $\mathcal{S}$
 - 10 Update best $\mathbf{c}_{\text{best}}$
- 11 **return** $\mathbf{c}_{\text{best}} \cdot (\mathbf{b} - \mathbf{a}) + \mathbf{a}$

DirectRectangle Data Structure

Each interval stores:

- $\mathbf{c}$: center point (in $[0, 1]^n$)
- $y = g(\mathbf{c})$: center function value
- $\mathbf{d} \in \mathbb{Z}^n$: number of divisions per dimension
- r : radius (half-diagonal)

Implementation Note

Efficient implementation uses a priority queue (min-heap) per distinct radius value r , sorted by $f(\mathbf{c})$. This allows $O(\log N)$ access to the best interval for each radius.

7.6 DIRECT — Python

Listing 6: DIRECT algorithm in Python

```
1 import numpy as np
2 from dataclasses import dataclass, field
3 from typing import List
4
5 @dataclass
6 class Rect:
7     c: np.ndarray      # center in unit hypercube
8     y: float           # f(c)
9     d: np.ndarray      # number of splits per dimension
10    r: float            # half-diagonal radius
11
12 def direct(f, a, b, k_max=20, r_min=1e-4):
13     """DIRECT algorithm for global optimisation over [a, b]."""
14     a, b = np.asarray(a, float), np.asarray(b, float)
15     n = len(a)
16     # Map to unit hypercube
17     g = lambda xp: f(xp * (b - a) + a)
18     c = np.full(n, 0.5)
19     rects = [Rect(c.copy(), g(c), np.zeros(n, int),
20                  np.sqrt(n) * 0.5)]
21     c_best = c.copy()
22     for _ in range(k_max):
23         # Find potentially optimal intervals (lower convex hull)
24         to_split = _potentially_optimal(rects, r_min)
25         new_rects = []
```

7.6 DIRECT — Worked Example

Example 7.1 — Flower Function on $[-1, 3] \times [-2, 1]$

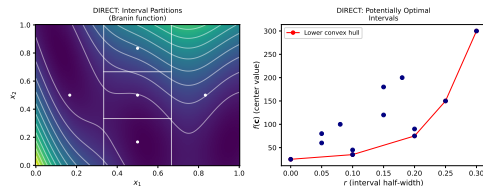
Normalised: $f(x'_1, x'_2) = \text{flower}(4x'_1 - 1, 3x'_2 - 2)$

Iteration 0: Sample at $[0.5, 0.5] \rightarrow f = 0.158$

After first split:

Int.	Center	Side lengths	r	$f(\mathbf{c})$
1	$[1/6, 3/6]$	$[1/3, 1]$	0.527	0.500
2	$[5/6, 3/6]$	$[1/3, 1]$	0.527	1.231
3	$[3/6, 3/6]$	$[1/3, 1/3]$	0.236	0.158
4	$[3/6, 1/6]$	$[1/3, 1/3]$	0.236	2.029
5	$[3/6, 5/6]$	$[1/3, 1/3]$	0.236	1.861

Next split: intervals 1 and 3 (on lower convex hull).



After several DIRECT iterations on Branin: left shows partitions, right shows $(r, f(\mathbf{c}))$ with lower convex hull in red.

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.7 Summary I

Summary of Direct Methods

- **Direct methods** rely solely on the objective function f — no derivatives required.

Cyclic Coordinate Search

- Optimises one coordinate at a time
- Simple but slow on diagonal valleys
- Acceleration: add $\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ step after each cycle

Generalized Pattern Search

- Generalises Hooke-Jeeves to arbitrary positive spanning sets $\mathcal{D}$
- Points lie on a scaled lattice
- Convergence guaranteed under mild conditions

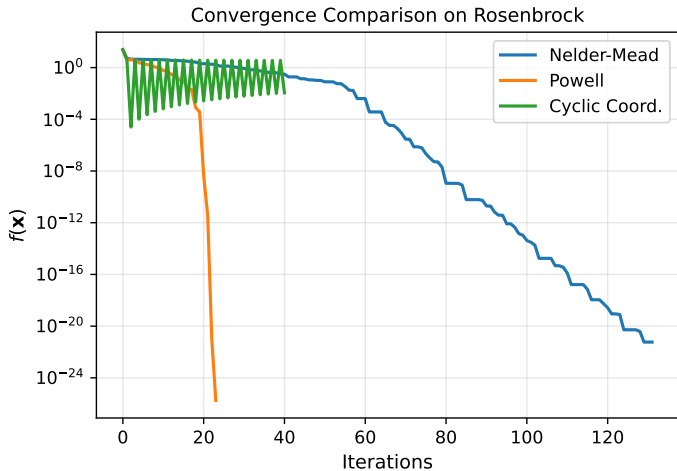
Powell's Method

- Builds conjugate search directions
- Faster convergence than cyclic search
- Quadratic convergence on quadratic objectives
- May lose conjugacy on non-quadratic f

Nelder-Mead

- Maintains and deforms a simplex of $n + 1$ points
- Four operations: reflection, expansion, contraction, shrinkage

7.7 Summary II



Convergence on Rosenbrock: Nelder-Mead and Powell converge faster than cyclic coordinate search.

7.7 Summary III

Method Comparison

Method	Global?	Evals/iter	Memory	Best for
Cyclic coord.	No	n	$O(n)$	Separable f
Powell	No	$n \cdot (ls)$	$O(n^2)$	Smooth f
Hooke-Jeeves	No	$2n$	$O(n)$	General f
GPS	No	$ \mathcal{D} $	$O(n \mathcal{D})$	Constrained lattice
Nelder-Mead	No	$1-n+1$	$O(n^2)$	Low-dim, smooth f
DIRECT	Yes	$2n$	$O(3^k)$	Bounded, Lipschitz f

7.8 Exercises I

Exercise 7.1

Design an objective function f and starting point such that Hooke-Jeeves fails to reduce the objective function. What does this imply about the algorithm's guarantees?

Hint: Consider a function whose minimum lies in a region that is never reached by coordinate-aligned steps of size α .

Exercise 7.2

Explain why cyclic coordinate search guarantees improvement after each full cycle provided the current point is not a minimum. What condition on f breaks this guarantee?

Hint: Think about a non-differentiable objective or one with flat regions. Moving in only increasing f would mean the method is at a local minimum.

7.8 Exercises II

Exercise 7.3

Give an example of a 2D objective function for which cyclic coordinate search with the standard $\pm \mathbf{e}_i$ directions will stall at a non-optimal point. Describe how GPS with a richer direction set $\mathcal{D}$ would succeed.

7.8 Exercises III

Exercise 7.4

Explain the difference between the divided rectangles algorithm (DIRECT) and the Nelder-Mead simplex method. Approximate how many function evaluations DIRECT requires for a 2-dimensional problem with 5 iterations.

Solution: Each iteration of DIRECT evaluates $2n = 4$ points per split. After $k = 5$ iterations the total number of rectangles grows at most as $O(3^k)$, though in practice far fewer are split each iteration.

7.8 Exercises IV

Exercise 7.5 — Computational

Find the optimal point of

$$f(x_1, x_2) = \sin\left(\frac{1}{2}x_1^2 - \frac{1}{4}x_2^2 + 3\right) \cos(2x_1 + 1 - e^{x_2})$$

over $x_1, x_2 \in [-2, 2]$ using:

- Ⓐ Nelder-Mead (from three different starting simplices)
- Ⓑ DIRECT (with `scipy.optimize.direct`)

Compare the solutions found. Which method is more reliable for this multimodal function?

Hint: Use `scipy.optimize.minimize(f, x0, method='Nelder-Mead')` and `scipy.optimize.direct(f, bounds=[(-2,2),(-2,2)])`.

References

-  M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026.
-  R. Hooke and T. A. Jeeves, “Direct Search Solution of Numerical and Statistical Problems,” *Journal of the ACM*, vol. 8, no. 2, pp. 212–229, 1961.
-  J. A. Nelder and R. Mead, “A Simplex Method for Function Minimization,” *The Computer Journal*, vol. 7, no. 4, pp. 308–313, 1965.
-  D. R. Jones, C. D. Perttunen, and B. E. Stuckman, “Lipschitzian Optimization Without the Lipschitz Constant,” *Journal of Optimization Theory and Applications*, vol. 79, no. 1, pp. 157–181, 1993.
-  R. G. Regis, “On the Properties of Positive Spanning Sets and Positive Bases,” *Optimization and Engineering*, vol. 17, no. 1, pp. 229–262, 2016.